Acta Psychologica 52 (1982) 107–123
North-Holland Publishing Company

STRUCTURAL FACTORS IN PATTERNED FINGER TAPPING *

Dirk-Jan POVEL and René COLLARD
University of Nijmegen, The Netherlands

Accepted July 1982

The relationship between the internal representation and the performance of serial tasks is studied. An experiment is reported in which subjects tapped as fast as possible serial 'finger patterns': sequences of taps made with different fingers. The patterns are assumed to be represented internally as hierarchies of operations. The latencies between successive taps are predicted from a production model – the Tree Traversal interpreter – which specifies the necessary actions to produce the finger pattern from a hierarchical memory representation. It was found that varying the internal representation of one and the same sequence of taps affects the performance in a predictable way. The relation of the proposed model to general theories of complex motor behavior is discussed.

The last decade has shown a growing interest in the study of complex motor tasks, like the tapping of morse codes, typing, speaking, piano playing and rhythm production (Klapp 1977; MacNeilage 1970; Martin 1972; Povel 1981; Shaffer 1978, 1981; Sternberg et al. 1978; Vorberg and Hambuch 1978). For a recent review of this work, see Keele (1981). Major issues in these investigations concern the nature of the internal representation of the task and the stages in the translation of this representation into action. Central in this research is the notion of a hierarchical organization of the serial task.

This paper studies the relation between the internal representation of a serial task and the actual performance of that task. The main purpose is to show experimentally that systematic variation of the internal

* We thank the technical staff for constructing the experimental setup, Marty Wansink for developing the necessary software and Cees Elzinga for his statistical advice. This research was supported in part by the Netherlands Organization for the Advancement of Pure Research (Z.W.O.).

Requests for reprints should be sent to D.J. Povel, Department of Experimental Psychology, P.O. Box 9104, 6500 HE Nijmegen, The Netherlands.

0001-6918/82/0000–0000/$02.75 © 1982 North-Holland

representation of one and the same task affects the latency profile of the movement pattern. A simple example may clarify the problem: suppose a person dialing the phone number 654456, which he/she has internally represented as if consisting of three groups, (65)(44)(56), while another person dialling the same number conceives it as (654)(456). Now it may be possible that the difference in the representations is reflected in the latency profiles of the actual dialling.

To study the role of the internal representation in the performance of serial tasks, we examined subjects who tapped repeatedly and as fast as possible sequences of taps performed with different fingers. Serial finger tapping seems a more suited task to study the present question than, say, speaking, typing or piano playing, for two reasons. First, there is an almost direct relation between each element in a numeric representation of a finger pattern in which the fingers are designated by the numbers 1 through 5. Thus, "3" always means: tap middle finger. In the other behaviors, the instruction to produce for instance an "a" may mean: speak the vowel "a", type the letter "a", or produce the tone "a"; in all three cases the required action has to be computed by determining the difference between the initial and target position (MacNeilage 1970).

Second, the structure of finger patterns is easier to manipulate systematically than the structure in sequences of speech sounds, typed letters or produced tones, being respectively of a linguistic or musical nature. It is true that these structures or contexts can be varied as witnessed by Shaffer's (1978) and Sternberg et al.'s (1978) research, but because of insufficient knowledge it is not as yet possible to make detailed predictions about the effect of the context structure upon relevant aspects of the responses. The structure in finger patterns, however, can be described by coding languages for the internal representation of patterned sequences as developed by Simon and Kotovsky (1963), Leeuwenberg (1971) and Restle (1970). Rosenbaum et al. (in press) reported an interesting study on finger tapping, in which he suggested that the latency profiles he observed may be explained by assuming a tree-like structural description of the finger pattern in memory. He studied essentially one basic pattern, realized in different finger orders, to control for possible effects of finger order on the latency profile. In the experiment to be reported, we systematically varied the internal representation of finger patterns, nevertheless controlling for finger order.

In the production of serial patterns one may distinguish two components (Greeno and Simon 1974): one component specifies the nature of the internal representation of the pattern, while the other describes the process that translates the representation into action. Therefore, we will first describe the coding model we adopted to represent the finger patterns and next the decoding component, the Tree Traversal interpreter (Collard and Povel 1982) will be discussed. Subsequently, it will be shown how predictions are derived from the compound model. At the end of this paper we will discuss how the present approach may fit into a broader framework of motor behavior.

Coding model of serial finger patterns

The above mentioned coding languages are all based upon the same elementary relationships, i.e. same, next and complement (Simon 1972), that appear to reflect the relations between the fingers of the hand in a most natural way. Subjects easily perform a sequence of taps from a numerical instruction, say (1 1 2 2 4 4 3 3), provided that consecutive numbers, 1, 2, 3,..., denote adjacent fingers. Such a serial instruction, however, is almost impossible to follow, if the numbers are assigned randomly to the fingers (e.g. 1 corresponds to middle finger, 2 to little finger, 3 to index, and 4 to ringfinger).

In order to represent formally the structure in finger patterns we have adopted Restle's classical TRM-model (Restle 1970, 1976; Simon 1972). The operations, T for transpose, R for repeat and M for mirror are defined by: $T(x) = xt(x)$, $R(x) = xr(x)$ and $M(x) = xm(x)$, where the lower case letters t, r and m denote transformations within an ordered alphabet. Given the alphabet [1 2 3 4], $t_1(1) = 2$, $t_2(2) = 3$, etc.; $r(1) = 1$, $r(2) = 2$, etc.; $m(1) = 4$, $m(2) = 3$, $m(3) = 2$ and $m(4) = 1$. Consequently, within the model the sequence (2 2) is represented by $R(2)$, (2 3) by $T_1(2)$, (4 2) by $T_{-2}(4)$ and (1 2 4 3) by $M(1\ 2)$, given the alphabet [1 2 3 4]. Throughout this article T_1 will simply be denoted by T. Operations may also be applied hiearchically: (1 2 4 3) can be coded by $M(T(1))$; (3 4 1 2) by $T_{-2}(T(3))$. This notation is easily extended to n-ary and to not strictly nested patterns (Restle 1970, 1973; Collard and Povel 1982). Thus, (1 2 3 4) can be represented by $3T(1)$, (1 2 4 4) by $T(1)$, $R(4)$ and (2 3 3 2 1 1) by 2, $R(3)$, 2, $R(1)$.

Decoding

So far we have introduced a notation for serial finger patterns (series of numbers) and a notation for the corresponding memory codes (*TRM*-codes). In this way structural information about a finger pattern may be encoded economically in the form of a hierarchical memory code. Conversely, when the pattern is to be reproduced, the memory code should be decoded again. This may seem a trivial point at first sight, since a formula like $T(R(T(1)))$ can easily be written out to $T(R(1\ 2))$ to $T(1\ 2\ 1\ 2)$ and finally to (1 2 1 2 2 3 2 3). Such a way of evaluating a code, however, requires a short term memory (or buffer) whose size equals half the sequence length. Greeno and Simon (1974) showed that there are other possible ways of interpreting TRM-codes which demand less memory costs but are equally capable of producing the sequence. In the present study we will apply the Tree Traversal interpreter that involves the notion of a structural tree (Restle 1970). A detailed formal description of this interpreter along with a comparative analysis to earlier models is presented in Collard and Povel (1982). An informal description of this interpreter will be given below.

A *TRM*-code may be depicted as a so-called structural tree that more clearly expresses the hierarchical structure of a serial pattern. Fig. 1 displays the structural tree corresponding to code $T(R(T(1)))$ of pattern (1 2 1 2 2 3 2 3). The interpreter operates by traversing the tree: starting at the leftmost element, the tree is traversed from left to right. In fig. 2 the same tree is shown, now with the corresponding transformations connected to the right oriented branches. Each time a transformation in the tree is encountered, it is applied straightforward when

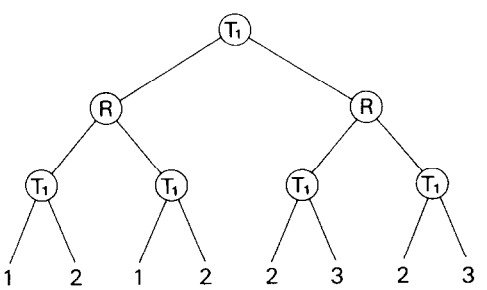

Fig. 1. Structural tree for code $T(R(T(1)))$ of sequence (1 2 1 2 2 3 2 3).

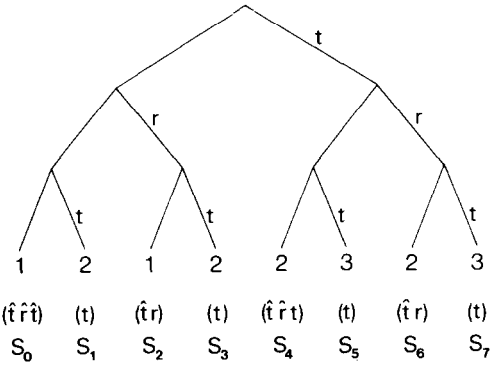

Fig. 2. Computational diagram of the Tree Traversal interpreter converting code $T(R(T(1)))$ to sequence (1 2 1 2 2 3 2 3). Each element S_i is computed from its immediate predecessor S_{i-1}, by traversing the tree. The transformations to be applied to the preceding element are given between parentheses ($\hat{t}$ and $\hat{r}$ are the inverse of t and r respectively).

moving downwards, and inversely when moving upwards. In fig. 2, the inverses of t (transpose) and r (repeat) are respectively denoted by $\hat{t}$ and $\hat{r}$ (note that $r = \hat{r}$).

When the rightmost element has been produced, the tree traversal can be continued via the top of the tree, rendering the first element of the sequence again. Thus, the Tree Traversal interpreter may decode in a repetitive fashion.

Predicting the timing profile

The performance profile associated with the Tree Traversal interpreter is derived from the computational cost to generate each element of the sequence from the memory code, and equals the number of transformations to be applied. Generally, the more levels in the tree have to be traversed, the more transformations are to be applied in order to generate the next element. Fig. 3 shows the performance profile for the sequence (1 2 1 2 2 3 2 3) with code $T(R(T(1)))$.

The Tree Traversal interpreter can also be conceived of as emitting a series of, possibly composite, transformations. Thus, at the lowest level not an element is emitted, but a description of the needed action to arrive at the next element. This conception is also displayed in fig. 2.

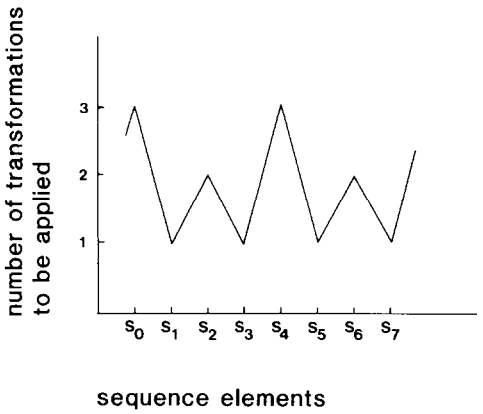

Fig. 3. Performance profile of the Tree Traversal interpreter for code $T(R(T(1)))$.

Such a way of operating seems most compatible with the subjective experience when repeatedly producing a finger pattern: once the movement has started, the actual control seems to be done in relational terms: one no longer realizes what fingers are actually involved.

The experiment

In order to study the role of the memory representation on the latency profile, the memory representation should be varied independently from other possible determinants. Of course, the actual finger to be tapped and the finger transition involved, are factors that will determine the interresponse intervals. Therefore, if systematic differences were found in the latency profiles of the patterns 1 2 3 4 and 1 3 2 4 (the numbers 1, 2, 3 and 4 respectively corresponding with index, middle finger, ring finger and little finger), it is impossible to decide what caused the different profiles: it may have been caused by the difference in finger order, by the difference in structure or by an interaction of these two factors. This problem was solved by constructing sets of stimuli that contained serial finger patterns with the same motor demands. The patterns within these sets are equal both with regard to the fingers used and the occurring finger transitions. This was accomplished by complementing one basic pattern with some of its cyclic permutations. An example of such a set contained the following three patterns: (1 2 3 3 2 1), (2 3 3 2 1 1) and (3 3 2 1 1 2). These patterns, being cyclic permutations of each other, have equal motor requirements when produced repeatedly, but may give rise to different structural descriptions. The first pattern: (1 2 3 3 2 1) will most likely be conceived of as consisting of two groups of three elements, the first of which can be coded as $2T(1)$, and the second group as the mirror image of the first, thus yielding the code: $M(2T(1))$. The second pattern, will probably

Table 1
The four sets of stimuli used in the experiment. Numbers indicate fingers: 1 = index, 2 = middle finger, 3 = ring finger and 4 = little finger. – = long interval; - = short interval.

A1.	3	2	1	2	3	4	$M(2T_{-1}(3))$	–S1-S2-S3–S4-S5-S6
A2.	2	1	2	3	4	3	$M(212)$	–S1-S2-S3–S4-S5-S6
A3.	1	2	3	4	3	2	$M(2T(1))$	–S1-S2-S3–S4-S5-S6
B1.	1	2	3	2	3	4	$T(2T(1))$	–S1-S2-S3–S4-S5-S6
B2.	2	3	4	1	2	3	$T_{-1}(2T(2))$	–S1-S2-S3–S4-S5-S6
B3.	2	3	2	3	4	1	$R(T(2)), M(4)$	–S1-S2–S3-S4–S5-S6
C1.	1	2	3	3	2	1	$M(2T(1))$	–S1-S2-S3–S4-S5-S6
C2.	3	3	2	1	1	2	$R(3), 2, R(1), 2$	–S1-S2–S3–S4-S5–S6
C3.	2	3	3	2	1	1	$2, R(3), 2, R(1)$	–S1–S2-S3–S4–S5-S6
D1.	2	4	3	4	2	1	no structure	–S1-S2-S3-S4-S5-S6
D2.	1	2	4	3	4	2	no structure	–S1-S2-S3-S4-S5-S6
D3.	3	4	2	1	2	4	no structure	–S1-S2-S3-S4-S5-S6

be coded completely different, namely as an alternation of element 2 with respectively two 3's ($R(3)$) and two 1's ($R(1)$), yielding the code 2, $R(3)$, 2, $R(1)$. The latter code is not hierarchical. The four sets of patterns used in the experiment are shown in table 1 along with their codes.

It will be observed that the four sets differ considerably with regard to the differentiation of the occurring structures (codes). This point will be further elaborated in the Results section. The predictions of the relative lengths of the intertap intervals, as inferred from the Tree Traversal interpreter, are presented in the last column of table 1. Note that the intervals are presented as latencies to the actual response, i.e. the interval between the preceding and the current response. The first interval represents the reaction time between the last element and the first element of the (repeatedly produced) pattern.

Method

Twenty subjects, all undergraduate students at Nijmegen University, participated in the experiment. Only subjects were selected that never played a musical instrument. Each trial consisted of the following stages: a pattern was shown to the subject as a sequence of numbers on a computer screen. The patterns were composed of the numbers 1 through 4, which corresponded respectively, with index, middle finger, ring finger and little finger. Next, the subject practised producing the pattern, that had to be tapped with the dominant hand, on four push buttons, one for each finger. The subjects practised until they could perform the pattern from memory; this took on the average about 2 minutes. Then the subject warned the experimenter that he/she was ready, whereafter the subject heard a ready signal followed after 1 sec by a "Go" signal upon which the subject had to start to tap the sequence as fast as possible, in a repeated

fashion. When the subject had tapped the pattern 6 times he/she received a stop signal. When the subject tapped a wrong button, an error signal sounded; in these cases the pattern was presented once more and had again to be produced 6 times. Only data from complete error-free trials were saved. The patterns were presented in random order, each subject receiving a different one. All subjects trained with 10 practise patterns before doing the actual experiment. Stimulus presentation and response collection was controlled by a PDP 11/03 computer.

Results

First we will present the tempo data for the patterns used in the experiment and then we will present the data concerning the latency profiles. In the computation of the data, all first realizations were disregarded; therefore, the data presented below are exclusively based on the 2nd through 6th realizations of the sequences.

Tempo data

Fig. 4 shows for each pattern the mean interresponse interval averaged over repetitions and subjects. This measure shows that the mean interresponse interval ranges between about 350 and 550 msec, indicating that the average subject tapped rather slow. The only distinct feature in the tempo data is that subjects produce the patterns of stimulus set four (which are the unstructured sequences; see below) significantly slower than the patterns in the other three sets. In fig. 4 the stimulus sets are separated by dashed vertical lines.

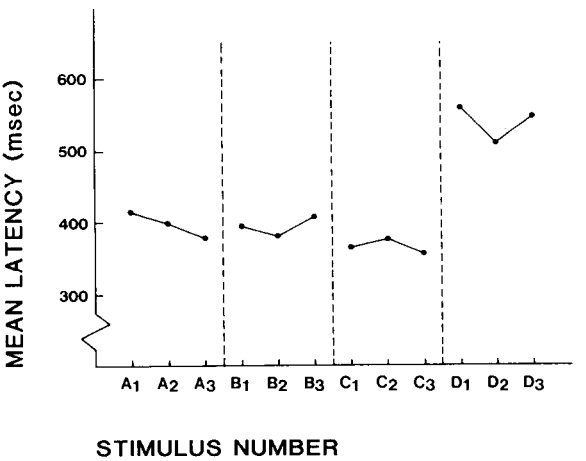

Fig. 4. Mean interresponse intervals (averaged over subjects and repetitions) for the 12 stimuli. Stimulus sets are indicated by dashed vertical lines.

Latency profiles

The data regarding the latency profiles will be treated separately for the four sets of stimuli, since, the effect of the memory code can only be studied within sets. One comment must be made about the way the data will be presented below. Before, it was stated that there are two possible determinants of the latency profile: the finger transitions involved and the memory code. Now suppose that the memory code is an unimportant factor and that the interresponse intervals are mainly determined by the motor demands to produce the actual finger transition. Then we would predict that if the observed timing profiles of the patterns within a set were superimposed on each other, such that the *finger transitions* were aligned, one would find identical profiles. If on the other hand, the memory code is the main factor in determining the interresponse intervals, we would predict that if the observed profiles of a set of patterns that share the same code, were superimposed such that the *codes* were aligned, one would find identical profiles. These two ways of presenting the data will further be called: finger-transitions aligned and code aligned.

The profiles shown below were obtained as follows: for each subject we calculated the mean interresponse intervals by averaging over the last 5 repetitions. This profile was subsequently transformed into ranks, such that the largest interval was assigned number 6 and the smallest interval number 1. The definitive profile was formed by the mean rank which was obtained by averaging over subjects. As a measure of the agreement among subjects the Coefficient of Concordance (W) was calculated for each pattern.

Stimulus set 1

This set consists of three patterns: A1 (3 2 1 2 3 4), A2 (2 1 2 3 4 3) and A3 (1 2 3 4 3 2). In each pattern the last three elements mirror the first three elements (see table 1). Therefore the three patterns in this set share the same structure. The timing profile as predicted from the Tree Traversal interpreter should therefore have the form: –S1-S2-S3–S4-S5-S6, wherre S_i indicates the ith element of the sequence and where "–" indicates a long interval and "-" a short interval. Fig. 5 shows the timing profiles of the three patterns displayed in the two ways explained above: finger-transitions aligned (left panel) and code aligned (right panel). The values of W calculated for the three patterns are respectively 0.57, 0.48 and 0.37, indicating that there is a fair agreement between subjects, be it less so for stimuli A2 and A3 than for stimulus A1, a point we will return to below. A comparison of the displays in fig. 5 shows that the profiles in the code aligned display are almost identical, whereas the profiles in the finger-transitions aligned display are quite dissimilar. Since in this stimulus set the codes for the three patterns are identical, this result convincingly supports the idea that the interresponse interval depends on the code and not on the actual finger transition. Moreover, it can be seen that the general form of the profiles follows the prediction specified above.

The lower W values for stimuli A2 and A3 may indicate that the patterns are ambiguous, or in other words, that an alternative structural description of these patterns is conceivable. Consider for instance stimulus A3: (1 2 3 4 3 2), this pattern, as assumed above, can be coded as $M(T(1))$, but also as $3T(1)$, $T_{-1}(3)$. According to the latter code the first four elements are conceived of as a group that can be described as a

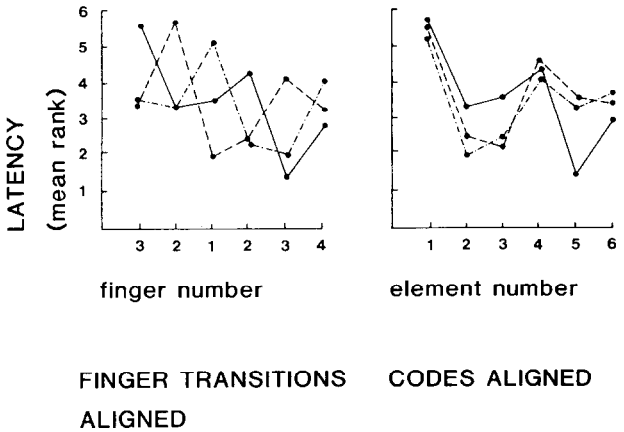

Fig. 5. Latency profiles for the three patterns of set 1: (3 2 1 2 3 4), (2 1 2 3 4 3) and (1 2 3 4 3 2), respectively indicated with a solid line, a dashed line and a dotted-dashed line. Left: finger transitions aligned. Right: code aligned.

sequence of ascending numbers, followed by a group of two elements. This alternative code is less economical than the first one because the two subgroups in the structure are not of equal size, and because the second group cannot be expressed as a transformation of the first. According to Leeuwenberg's (1971) "minimum principle", subjects will prefer the most economical structural description. That code is the one given in table 1. If we inspect the individual profiles, we do find that 13 subjects clearly show a timing profile corresponding to the $M(2T(1))$ code, while 7 subjects apparently applied the second code. Fig. 6 shows the average profiles of the two groups of subjects [1]. It is clearly seen that the profiles become much more pronounced, which is also reflected in the higher W values we now find: 0.64 and 0.77 respectively.

The same reasoning can be applied to stimulus A2: (2 1 2 3 4 3). An alternative code for this pattern is: a group of four elements consisting of a 2 alternated respectively by a 1 and a 3, which is followed by $T_{-1}(4)$. This code yields the following timing profile: –S1-S2-S3-S4–S5-S6. Upon inspection of the individual profiles we now find 6 profiles that obey this code, while the others follow the more economical code. For stimulus A1 an alternative code is not easily conceivable. Inspection of the individual profiles indeed reveals that in all of them the first and fourth latency are the two largest.

Thus, for the first stimulus set, we may conclude that all observed profiles fit the view that the interresponse intervals depend on the nature of the code, especially if we take into account the ambiguity in two of the stimuli.

[1]For the sake of clearness, in figs. 6 through 8 finger number is indicated on the X-axis, not element number. It should be kept in mind, however, that the latency profile is supposed to depend on the code from which the consecutive elements are produced, rather than on the finger transitions involved.

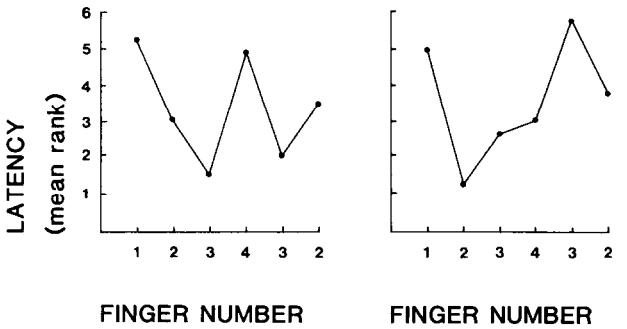

Fig. 6. Latency profiles of pattern A3 of set 1: (1 2 3 4 3 2). Left: subjects who apply the code $M(2T(1))$. Right: subjects who apply the code $3T(1)$, $T_{-1}(3)$.

Stimulus set 2

The patterns in this set are: B1 (1 2 3 2 3 4), B2 (2 3 4 1 2 3) and B3 (2 3 2 3 4 1). The first two patterns in this set have similar codes: $T(2T(1))$ and $T_{-1}(2T(2))$ while the third pattern has a completely different code: $R(T(2))$, $M(4)$. According to the Tree Traversal interpreter, stimuli 1 and 2 are predicted to show the profile –S1-S2-S3–S4-S5-S6 whereas pattern 3 should show profile –S1-S2–S3-S4–S5-S6. The W values obtained for the three profiles are 0.67, 0.69 and 0.52 respectively.

Fig. 7, presents the latency profiles associated with the three patterns. The profiles agree quite well with the predictions: it is clear that the patterns 1 and 2 are subdivided into two groups of three elements, whereas pattern 3 is subdivided into 3 groups of 2 elements.

Stimulus set 3

The patterns in this set are C1 (1 2 3 3 2 1), C2 (3 3 2 1 1 2) and C3 (2 3 3 2 1 1). This is the only set built from a pattern in which the same finger must be tapped twice

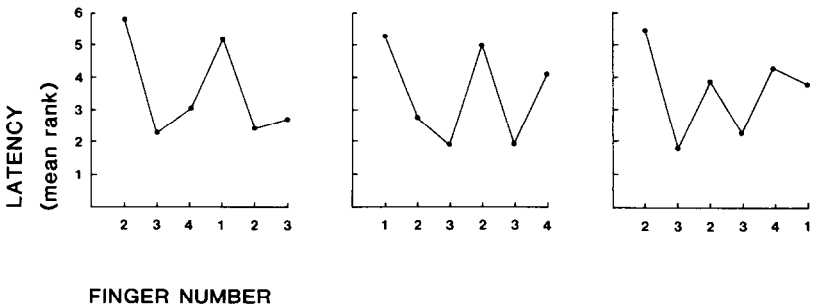

Fig. 7. The three latency profiles observed for the patterns of stimulus set 2: (1 2 3 2 3 4), (2 3 4 1 2 3) and (2 3 2 3 4 1).

in succession. When constructing the stimulus sets we tried to avoid the occurrence of such a repetition because it is known that the interresponse interval is substantially longer when two consecutive taps have to be performed by the same finger, as compared with two taps performed by two different fingers. Nevertheless, we decided to include this set because it contained some patterns with nicely contrasting codes, namely: $M(2T(1))$; $R(3)$, 2, $R(1)$, 2 and 2, $R(3)$, 2, $R(1)$. The predicted latency profiles associated with the three codes are respectively: –S1-S2-S3–S4-S5-S6, –S1-S2–S3–S4-S5–S6 and –S1–S2-S3–S4–S5-S6. The W values calculated for the three patterns are respectively: 0.23, 0.50 and 0.61. The low W value for the first pattern (C1) suggests ambiguity. Indeed it could be inferred from the individual profiles that some subjects code this pattern as: $T(1)$, $R(3)$, $T_{-1}(1)$; which shows that repetition is a striking feature. For the rest it can be seen, that the observed profiles very well agree with the theoretically predicted profiles (see fig. 8). This, again, supports the hypothesis that the memory code plays a decisive role in determining the interresponse intervals.

We will now look at the three profiles separately. The first pattern (C1) with code $M(2T(1))$ shows two groups of 3 elements. Note that the latency before the second tap of the repeated middle finger (3) as well as the latency before the second tap of the repeated index (1) are longer than the latency to the first tap of these pairs. This finding is in agreement with the above mentioned finding that tapping with the same finger is done slower than with two different fingers. This characteristic, however, is also in agreement with the predictions from our model. Now consider stimuli C2 and C3: the intervals between the taps made with the same fingers (3 3) and (1 1) are now consequently the shortest intervals occurring in the two profiles. This clearly demonstrates that the way the two identical finger taps are conceptualized, determines the latency: if the second of the two taps made with the same finger is conceived as being a repetition of the preceding one (stimulus C2 and C3) the interresponse time is very short, but if the second tap of these pairs is not conceived as a repetition (stimulus C1) the observed interresponse intervals are the longest occurring in that profile. Apparently, the memory code even overrules the normally found lengthening of the interval between two taps made with one and the same finger.

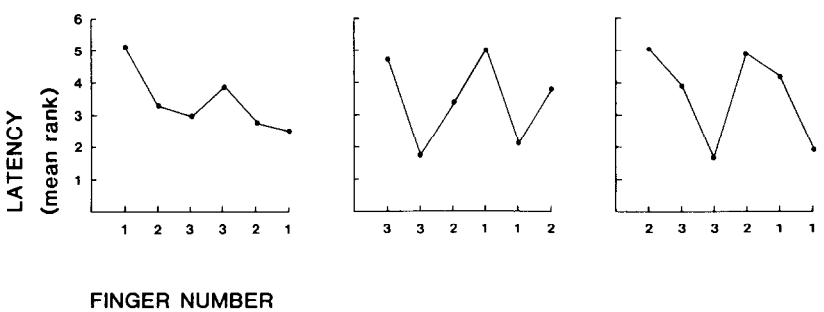

Fig. 8. The latency profiles observed for the patterns of stimulus set 3: (1 2 3 3 2 1), (3 3 2 1 1 2) and (2 3 3 2 1 1).

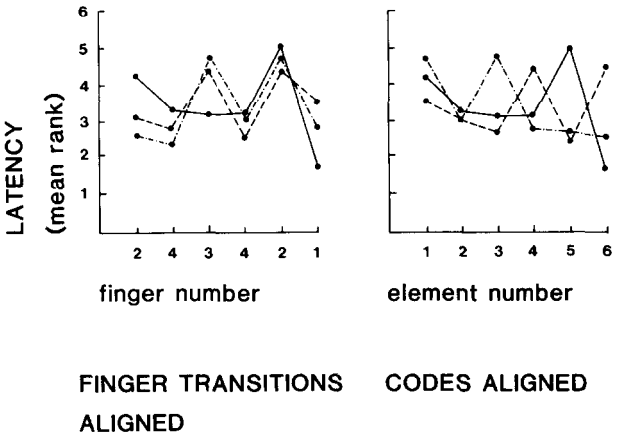

Fig. 9. The latency profiles observed for the patterns of stimulus set 4: (2 4 3 4 2 1), (1 2 4 3 4 2) and (3 4 2 1 2 4), respectively indicated with a solid line, a dashed line and a dotted-dashed line. Left: finger transitions aligned. Right: stimuli aligned.

Stimulus set 4

The patterns in this set are: D1 (2 4 3 4 2 1), D2 (1 2 4 3 4 2) and D3 (3 4 2 1 2 4). This set is special in as far as the three patterns it contains seem to possess no structure. It is indeed very difficult or even impossible to find a rule to describe the structure in these patterns in an efficient way. We therefore assume that these patterns are stored in memory as a series of unstructured elements, and not in the form of a code. Consequently, in the production of these sequences coding and decoding mechanisms should not come into play. We therefore predict that if any differentiation is found in the profiles, it is caused by differences in the motor requirements to realise a particular finger transition. So, for this set, we expect to find the reverse situation as the one observed in the first set of stimuli: now the profiles should coincide if we align the finger transitions and not coincide if we align the stimuli as they were presented. Fig. 9 clearly shows that these predictions are confirmed.

The W values found for the three profiles are: 0.39, 0.21 and 0.32 respectively, which is lower than the values we calculated for the unambiguously structured sequences. The finger transitions aligned display shows that the transitions 4 to 3 and 4 to 2 are apparently the most difficult ones to be performed. The fact that the profile of stimulus D1 in the finger-aligned display does not completely follow the other profiles, may indicate that some subjects still use a partial coding.

Discussion

From the data presented it may be concluded that the latency profile of a tapped finger pattern is largely determined by the internal representa-

tion of that pattern. It was shown for four different stimulus sets, with finger order kept constant within each set, that the observed latency profiles do agree with the predictions made on the basis of the *TRM*-coding model and the Tree Traversal interpreter.

Here we will consider how the Tree Traversal interpreter may fit in a more general framework of motor programming. Basically, three different views have been proposed: the Preprogramming model, the Clock model and the On-line programming model, which are schematically reproduced in fig. 10.

1. The Pre-programming model. This model has been proposed as the "sequence-preparation hypothesis" by Sternberg et al. (1978) to explain findings related to the reaction times and latency profiles of sequences of spoken words and typed letters. Here, we do not go into all assumptions underlying the model, but only refer to the notion of the buffer in the model. The authors assume that the so-called "motor-program" buffer contains (a program consisting of) a set of linked subroutines, one for each unit in the serial task. The code in this buffer "... is not sensitive to factors such as familiarity of the response, similarity or identity among its elements...". In other words the buffer contains a chain of unrelated subroutines. According to the Pre-programming model, the latency to a response in a serial task, is de-

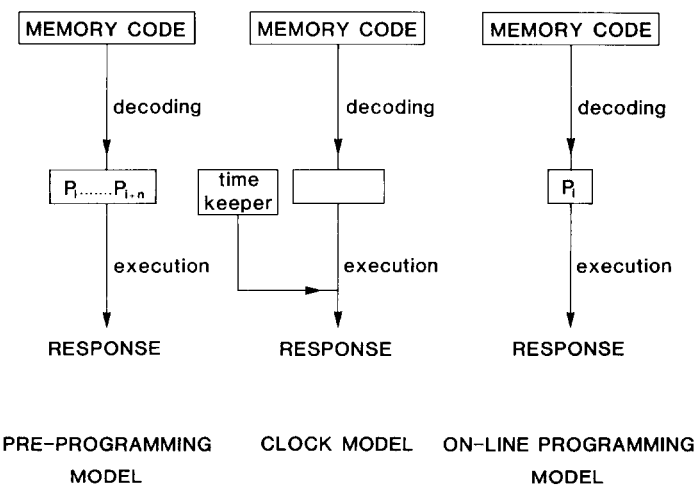

Fig. 10. Three models for the performance of serial tasks. P_i = motor instruction (program) to produce Response (i). See text for description of the models.

termined by the number of subprograms in the buffer on the one hand and by the complexity of the subprogram actually to be run on the other.

2. The Clock model. This model based on the time-keeper model of Wing and Kristofferson (1973) for the production of sequences of equally spaced taps, has been proposed by Shaffer (1978, 1981) for typewriting and piano playing. The model is more complex than the other two models discussed in this section as it assumes an independent time-keeper mechanism that determines the onset of each response. According to this model, the interresponse interval will not reflect any characteristics of the memory code or of the decoding process, since it is the time-keeper mechanism that determines the timing of the successive events. Shaffer (1978) did present data from one subject, who typed nine times a sequence of words whose letters were on alternate hands, that showed a negative auto-correlation at lag 1 as predicted by Wing and Kristofferson's model. Another finding of his, namely that typists show different interresponse interval profiles for the same trigram in different contexts, is reason for him to assume that "a skilled typist can program a timing pattern for letter sequences" (1978: 336).

How reasonable is it to suppose that subjects program a timing pattern when generating tasks like typing, tapping and piano playing (Shaffer 1978; Terzuolo and Viviani 1980)? It may be helpful to introduce here a distinction between tasks with and without intrinsic timing constraints. Intuitively, there is in this respect a great difference between piano playing and typing. In piano playing, besides hitting the correct keys, the main task is to produce a specific timing pattern. In typing, however, the only practical timing constraint seems to be that the work should be done as fast as possible.

3. The On-line programming model. The latter consideration leads to a more parsimonious explanation of the timing profiles found in typing, which assumes that the duration of the intervals between two responses is the result of the momentary load of the task. In this view a relatively long interresponse interval is predicted if the necessary preparations, whatever their nature, to make a response are relatively complex. The most important feature of the On-line programming model is the assumption that not more than one response at a time is prepared while the evaluation of the memory code is only continued when the preceding response has been executed or is in the course of being executed. According to this model, the interresponse time is determined apart

from the motor demands, by the computation time needed to evaluate the memory code. Within this model, therefore, characteristics from the code may be reflected in the timing characteristics of the produced task.

The Tree Traversal interpreter employed in this paper, may either be conceived of as the decoding component of the Pre-programming model or as that of the On-line programming model, since in both conceptions it yields the same predictions. If the interpreter operates within a Pre-programming model, the predictions of the code-dependent latencies are based on the compound description of the necessary action to produce the current response. For example in fig. 2, the description to generate the fifth element (S4) reads: $\hat{t}\hat{r}t$. If, on the other hand, the interpreter is seen as the decoding component in the On-line programming model, the latencies are predicted by the computation time to generate the current response. The On-line programming model is more parsimonious than the Preprogramming model, since it does not require an additional buffer.

References

Collard, R.F.A. and D.J. Povel, 1982. Theory of serial pattern production: tree traversals. Psychological Review 89, 693–707.

Greeno, J.G. and H.A. Simon, 1974. Processes for sequence production. Psychological Review 81, 187–198.

Keele, S.W., 1981. 'Behavioral analysis of movement'. In: V.B. Brooks (ed.), Handbook of physiology: Section 1: The nervous system. Volume II. Motor control, Part 2. Baltimore, MD: American Physiological Society.

Klapp, S.T., 1977. 'Reaction time analysis of programmed control'. In: R. Hutton (ed.), Exercise and sport sciences review, Vol. 5. Santa Barbara, CA: Journal Publishing Affiliates.

Leeuwenberg, E.L.J., 1971. A perceptual coding language for visual and auditory patterns. American Journal of Psychology 84, 307–349.

MacNeilage, P.F., 1970. Motor control of serial ordering of speech. Psychological Review 77, 182–196.

Martin, J.G., 1972. Rhythmic (hierarchical) versus serial structure in speech and other behavior. Psychological Review 79, 487–509.

Povel, D.J., 1981. Internal representation of simple temporal patterns. Journal of Experimental Psychology: Human Perception and Performance 7, 3–18.

Restle, F., 1970. Theory of serial pattern learning. Psychological Review 77, 481–495.

Restle, F., 1973. Serial pattern learning: higher order transitions. Journal of Experimental Psychology 99, 61–69.

Restle, F., 1976. Structural ambiguity in serial pattern learning. Cognitive Psychology 8, 357–381.

Rosenbaum, D.A., S.B. Kenny and M.A. Derr, in press. Hierarchical control of rapid movement sequences. Journal of Experimental Psychology: Human Perception and Performance.

Shaffer, L.H., 1978. Timing in motor programming of typing. Quarterly Journal of Experimental Psychology 30, 333–345.

Shaffer, L.H., 1981. Performances of Chopin, Bach and Bartok: studies in motor programming. Cognitive Psychology 13, 326–376.

Simon, H.A., 1972. Complexity and the representation of patterned sequences of symbols. Psychological Review 79, 369–382.

Simon, H.A. and K. Kotovsky, 1963. Human acquisition of concepts for sequential patterns. Psychological Review 70, 295–306.

Sternberg, S., S. Monsell, R.L. Knoll and C.E. Wright, 1978. 'The latency and duration of rapid movement sequences: comparisons of speech and typewriting'. In: G.E. Stelmach (ed.), Information processing in motor control and learning. New York: Academic Press.

Terzuolo, C.A. and P. Viviani, 1980. Determinants and characteristics of motor patterns used for typing. Neuroscience 5, 1085–1103.

Vorberg, D. and R. Hambuch, 1978. 'On the temporal control of rhythmic performance'. In: J. Requin (ed.), Attention and performance, VIII. Hillsdale, NJ: Erlbaum.

Wing, A.M. and A.B. Kristofferson, 1973. Response delays and the timing of discrete motor responses. Perception & Psychophysics 14, 5–12.